

# MediAssist AI

## Healthcare Workflow Support System

---

AI for Bharat Hackathon 2026 | Healthcare & Life Sciences Track

---



### Team Cyber Scorpion

Team Lead: Bhavik Patel

Team Member: Hem Bhavsar

March 2026

# 1. Executive Summary

---

MediAssist AI is an AI-powered clinical document processing prototype that addresses a critical bottleneck in healthcare — the time clinicians spend translating complex medical records into patient-friendly language. Built on AWS serverless architecture with a local LLM, MediAssist AI provides:

- Automated professional clinical summaries from uploaded documents
- Plain-language patient explanations generated by Ollama LLM
- Human-in-the-loop approval workflow ensuring doctor oversight
- Full audit trail for regulatory compliance
- Support for PDF, DOCX, images (JPG/PNG), and scanned documents via OCR

 **Important:** This system processes synthetic data only and is a demonstration prototype. It does not provide medical diagnoses or treatment recommendations.

## 2. Problem Statement

---

### The Challenge

Healthcare professionals in India and globally face a persistent problem: complex clinical documentation is difficult for patients to understand, leading to poor health literacy, medication non-compliance, and worse outcomes.

| Challenge                                   | Impact                                              |
|---------------------------------------------|-----------------------------------------------------|
| Clinical language is highly technical       | Patients cannot understand their own health records |
| Doctors lack time for detailed explanations | Average consultation: 2–5 minutes                   |
| Manual summarization is error-prone         | Risk of miscommunication and harm                   |
| No audit trail for AI-generated content     | Compliance and liability concerns                   |
| Scanned/image documents are inaccessible    | Large volume of legacy paper records in India       |

### The Opportunity

India has 1 doctor per 834 patients (WHO standard: 1 per 1000). AI-assisted documentation can help stretch clinical capacity while keeping humans firmly in control of medical decisions.

### 3. Our Solution

MediAssist AI provides a 4-step workflow for clinical document processing:

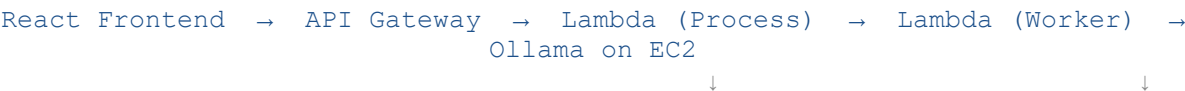
| Step      | Description                                                     |
|-----------|-----------------------------------------------------------------|
| ① Upload  | Clinician uploads PDF, DOCX, image or text document             |
| ② Process | Document sent to AWS Lambda, text extracted, AI triggered async |
| ③ Review  | Doctor reviews AI summary, edits if needed, approves release    |
| ④ Export  | Approved explanation exported with full audit trail             |

#### Key Differentiators

- No API timeout issues — async worker pattern handles Ollama's 60-90s processing time
- Privacy-first — LLM runs on private EC2, no data sent to third-party APIs
- OCR support — Amazon Textract handles scanned PDFs and photos of documents
- Human oversight — AI cannot release content without doctor approval
- Compliance-ready — every action logged with timestamp and reviewer ID

### 4. System Architecture

#### Architecture Diagram



DynamoDB

S3 Storage

## Why Async Pattern?

**Problem:** AWS API Gateway has a hard 29-second timeout. Ollama LLM takes 60-90 seconds to process clinical documents.

**Solution:** Process Lambda returns doc\_id instantly (<1s). Worker Lambda runs Ollama in background (600s timeout). React frontend polls /status every 5 seconds.

## AWS Services Used

| Service            | Purpose                                                      |
|--------------------|--------------------------------------------------------------|
| AWS Lambda (×5)    | Serverless compute — Process, Worker, Status, Approve, Audit |
| Amazon API Gateway | HTTP API with CORS — 4 routes                                |
| Amazon DynamoDB    | Results, Users, AuditLog tables (PAY_PER_REQUEST)            |
| Amazon S3          | Document storage + React frontend hosting                    |
| Amazon Textract    | OCR for scanned PDFs and image uploads                       |
| Amazon CloudWatch  | Live monitoring dashboard for all Lambda functions           |
| AWS IAM            | Least-privilege roles for all services                       |

## Lambda Functions

| Function           | Timeout / Memory / Role                                             |
|--------------------|---------------------------------------------------------------------|
| MediAssist-Process | 30s / 512MB — Receives upload, extracts text, triggers Worker async |
| MediAssist-Worker  | 600s / 512MB — Calls Ollama, updates DynamoDB with AI result        |
| MediAssist-Status  | 30s / 256MB — Polled by React every 5s to check completion          |
| MediAssist-Approve | 30s / 256MB — Doctor approves/edits summary, updates status         |

|                  |                                                 |
|------------------|-------------------------------------------------|
| MediAssist-Audit | 30s / 256MB — Returns full compliance audit log |
|------------------|-------------------------------------------------|

## 5. Technology Stack

### Frontend

| Technology | Usage |
|------------|-------|
|------------|-------|

|                   |                                                   |
|-------------------|---------------------------------------------------|
| React 19.2 + Vite | Single-page application with fast HMR development |
| TailwindCSS       | Responsive UI styling                             |
| React Context API | Global state management across 4 workflow panels  |
| Fetch API         | REST calls + polling pattern for async status     |

## Backend

| Technology           | Usage                                    |
|----------------------|------------------------------------------|
| Python 3.11          | All Lambda function runtime              |
| PyPDF2               | Text extraction from PDF files           |
| python-docx          | Text extraction from DOCX files          |
| boto3                | AWS SDK — DynamoDB, S3, Lambda, Textract |
| Ollama + llama3.2:3b | Local LLM on EC2 t3.large (8GB RAM)      |

## Document Processing Support

| File Type           | Extraction Method                    |
|---------------------|--------------------------------------|
| .txt                | Direct UTF-8 decode                  |
| .pdf (text-based)   | PyPDF2 page extraction               |
| .pdf (scanned)      | Auto-fallback to Amazon Textract OCR |
| .docx               | python-docx paragraph extraction     |
| .jpg / .png / .tiff | Amazon Textract OCR                  |
| .bmp / .webp        | Amazon Textract OCR                  |

## 6. Responsible AI Approach

| Principle       | How We Implement It                                                      |
|-----------------|--------------------------------------------------------------------------|
| Human Oversight | Every AI output requires explicit doctor approval before patient sees it |

|                    |                                                                      |
|--------------------|----------------------------------------------------------------------|
| Transparency       | Confidence scores (High/Medium/Low) shown for every summary          |
| Safety Constraints | LLM prompt explicitly prevents diagnostic/treatment recommendations  |
| Audit Trail        | Every action (upload, generate, edit, approve) logged with timestamp |
| Data Privacy       | LLM runs on private EC2 — no clinical data sent to third-party APIs  |
| Bias Awareness     | Synthetic data only — no real patient data processed in prototype    |

**Core Principle:** The AI assists, the doctor decides. MediAssist AI cannot release any content to patients without explicit human approval.

---

## 7. API Reference

---

| Endpoint      | Description                                                      |
|---------------|------------------------------------------------------------------|
| POST /process | Upload document → returns {doc_id, status: PROCESSING} instantly |

|                        |                                                            |
|------------------------|------------------------------------------------------------|
| GET /status?doc_id=xxx | Poll for AI result — returns PROCESSING / PENDING / FAILED |
| POST /approve          | Doctor approves/edits summary → status becomes APPROVED    |
| GET /audit?doc_id=xxx  | Returns full chronological audit log for compliance        |

API Gateway

| Property | Value                                                        |
|----------|--------------------------------------------------------------|
| Type     | HTTP API v2 (not REST API)                                   |
| CORS     | AllowOrigins: *, Methods: GET, POST, OPTIONS                 |
| Region   | ap-south-1 (Mumbai)                                          |
| Base URL | https://68nnaciiv6.execute-api.ap-south-1.amazonaws.com/prod |
| Auth     | None (demo) — add Cognito for production                     |

8. Deployment

Infrastructure (AWS — ap-south-1 Mumbai)

| Resource | Details |
|----------|---------|
|----------|---------|

|                 |                                                                   |
|-----------------|-------------------------------------------------------------------|
| EC2 Instance    | t3.large, 8GB RAM — running Ollama + llama3.2:3b                  |
| S3 Buckets      | mediassistai-documents (storage), mediassistai-frontend (hosting) |
| DynamoDB Tables | MediAssist-Results, MediAssist-Users, MediAssist-AuditLog         |
| IAM Role        | MediAssist-Lambda-Role with S3, DynamoDB, Textract, Lambda invoke |

## Deployment Commands

```
# Full deployment
cd PrototypeDemo/infra && bash deploy.sh
# CloudWatch dashboard
bash setup_dashboard.sh
# Pre-warm Ollama before demo
curl http://3.109.55.138:11434/api/generate -d
'{"model":"llama3.2:3b","prompt":"hello","stream":false}'
```

---

## 9. Limitations & Future Work

### Current Limitations (Prototype)

- No user authentication — add AWS Cognito for production

- English language only — multilingual support needed for AI for Bharat
- Processing time: 60-90 seconds — upgrade EC2 to GPU instance for <10s
- Scanned image quality affects OCR accuracy
- Not HIPAA certified — prototype uses synthetic data only

### Roadmap for Production

| Feature                                       | Priority |
|-----------------------------------------------|----------|
| Multilingual support (Hindi, Tamil, Gujarati) | High     |
| GPU EC2 for faster inference                  | High     |
| AWS Cognito authentication                    | High     |
| HL7 FHIR integration                          | Medium   |
| Mobile app (React Native)                     | Medium   |
| Real-time collaboration for doctors           | Low      |

## 10. Team

| Detail    | Information      |
|-----------|------------------|
| Team Name | Cyber Scorpion 🦂 |
| Team Lead | Bhavik Patel     |

|             |                                                                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Team Member | Hem Bhavsar                                                                                                                         |
| Hackathon   | AI for Bharat 2026 (Powered by AWS)                                                                                                 |
| Track       | Healthcare & Life Sciences                                                                                                          |
| Submission  | March 2026                                                                                                                          |
| GitHub      | <a href="https://github.com/BhavikPatel/AI-for-Healthcare-Life-Sciences">github.com/BhavikPatel/AI-for-Healthcare-Life-Sciences</a> |

 **This system uses synthetic data only. Not for clinical use.**